

AgroAidApp - Guia de codigo manual

Dias 3 a 6. Archivo pensado para copiar a mano: cada bloque indica la ruta, si hay que crear o reemplazar, y el codigo completo sugerido.

Contexto usado: app Expo Router con alias @/*, servicios en services/, hooks en hooks/, colores en constants/colors.ts y backend actual con /api/consulta, /api/branding y /api/historial.

Día	Trabajo
3	Tabs, pantallas base y branding del tenant.
4	Servicio/hook de consultas y primer flujo funcional.
5	Componentes reutilizables para consulta y UI final del chat.
6	Servicio y pantalla de historial.

DIA 3

Crear la carpeta app/(tabs). Modificar el layout raiz para agregar BrandingProvider. Nota: en Expo Router el grupo (tabs) no crea una URL /tabs; cuando ajustes redirects usa '/(tabs)' si hace falta.

AgroAidApp/app/_layout.tsx - MODIFICAR - reemplazar contenido completo

```
import { DarkTheme, DefaultTheme, ThemeProvider } from '@react-navigation/native';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import type { ReactNode } from 'react';
import { useEffect } from 'react';
import { useColorScheme } from 'react-native';

import { BrandingProvider } from '@/hooks/useBranding';
import { AuthProvider } from '@/hooks/useAuth';

SplashScreen.preventAutoHideAsync();

function AuthGuard({ children }: { children: ReactNode }) {
  return children;
}

export default function RootLayout() {
  const colorScheme = useColorScheme();

  useEffect(() => {
    SplashScreen.hideAsync();
  }, []);

  return (
    <ThemeProvider value={colorScheme === 'dark' ? DarkTheme : DefaultTheme}>
      <AuthProvider>
        <BrandingProvider>
          <AuthGuard>
            <Stack screenOptions={{ headerShown: false }} />
          </AuthGuard>
        </BrandingProvider>
      </AuthProvider>
      <StatusBar style="auto" />
    </ThemeProvider>
  );
}
```

AgroAidApp/hooks/useBranding.ts - CREAR

```
import {
  createContext,
  createElement,
  type ReactNode,
```

```
    useCallback,
    useContext,
    useEffect,
    useMemo,
    useState
  } from 'react';

import { colors } from '@/constants/colors';
import { apiFetch } from '@services/api';

export type Branding = {
  logo_url: string | null;
  primary_color: string;
  accent_color: string;
  app_name: string;
  footer_text: string;
};

const defaultBranding: Branding = {
  logo_url: null,
  primary_color: colors.brand.primary,
  accent_color: colors.brand.secondary,
  app_name: 'AgroAid',
  footer_text: 'AgroAid - Seguridad agricola integral'
};

type BrandingContextValue = {
  branding: Branding;
  isLoading: boolean;
  reloadBranding: (slug?: string) => Promise<void>;
};

const BrandingContext = createContext<BrandingContextValue | null>(null);

function normalizeBranding(data: Partial<Branding>): Branding {
  return {
    logo_url: data.logo_url ?? defaultBranding.logo_url,
    primary_color: data.primary_color ?? defaultBranding.primary_color,
    accent_color: data.accent_color ?? defaultBranding.accent_color,
    app_name: data.app_name ?? defaultBranding.app_name,
    footer_text: data.footer_text ?? defaultBranding.footer_text
  };
}

export function BrandingProvider({ children }: { children: ReactNode }) {
  const [branding, setBranding] = useState<Branding>(defaultBranding);
  const [isLoading, setIsLoading] = useState(true);

  const reloadBranding = useCallback(async (slug = 'default') => {
    setIsLoading(true);
    try {
      const params = new URLSearchParams({ slug });
      const data = await apiFetch<Branding>(`/api/branding?${params}`, {
        skipAuth: true
      });
      setBranding(normalizeBranding(data));
    }
  });
}
```

AgroAidApp - Dias 3 a 6

```
    } catch {
      setBranding(defaultBranding);
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    reloadBranding();
  }, [reloadBranding]);

  const value = useMemo<BrandingContextValue>(
    () => ({ branding, isLoading, reloadBranding }),
    [branding, isLoading, reloadBranding]
  );

  return createElement(BrandingContext.Provider, { value }, children);
}

export function useBranding() {
  const context = useContext(BrandingContext);

  if (!context) {
    throw new Error('useBranding debe usarse dentro de BrandingProvider');
  }

  return context;
}
```

AgroAidApp/app/(tabs)/_layout.tsx - CREAR

```
import { Ionicons } from '@expo/vector-icons';
import { Tabs } from 'expo-router';

import { colors } from '@constants/colors';
import { useBranding } from '@hooks/useBranding';

export default function TabsLayout() {
  const { branding } = useBranding();
  const activeColor = branding.primary_color || colors.brand.primary;

  return (
    <Tabs
      screenOptions={{
        headerStyle: { backgroundColor: colors.surface.card },
        headerTintColor: colors.text.primary,
        tabBarActiveTintColor: activeColor,
        tabBarInactiveTintColor: colors.text.secondary,
        tabBarStyle: {
          backgroundColor: colors.surface.card,
          borderTopColor: colors.surface.border
        }
      }}
    >
    <Tabs.Screen
```

```

    name="index"
    options={{
      title: 'Consulta',
      tabBarIcon: ({ color, size }) => (
        <Ionicons name="chatbubbles-outline" color={color} size={size} />
      )
    }}
  />
<Tabs.Screen
  name="mapa"
  options={{
    title: 'Mapa',
    tabBarIcon: ({ color, size }) => (
      <Ionicons name="map-outline" color={color} size={size} />
    )
  }}
/>
<Tabs.Screen
  name="historial"
  options={{
    title: 'Historial',
    tabBarIcon: ({ color, size }) => (
      <Ionicons name="time-outline" color={color} size={size} />
    )
  }}
/>
<Tabs.Screen
  name="recetas"
  options={{
    title: 'Recetas',
    tabBarIcon: ({ color, size }) => (
      <Ionicons name="document-text-outline" color={color} size={size} />
    )
  }}
/>
<Tabs.Screen
  name="perfil"
  options={{
    title: 'Perfil',
    tabBarIcon: ({ color, size }) => (
      <Ionicons name="person-outline" color={color} size={size} />
    )
  }}
/>
</Tabs>
);
}

```

AgroAidApp/app/(tabs)/index.tsx - CREAM version inicial

```

import { StyleSheet, Text, TextInput, View } from 'react-native';

import { colors } from '@constants/colors';
import { useBranding } from '@hooks/useBranding';
export default function ConsultaTab() {

```

AgroAidApp - Dias 3 a 6

```
const { branding } = useBranding();

return (
  <View style={styles.screen}>
    <Text style={styles.title}>{branding.app_name}</Text>
    <Text style={styles.subtitle}>Consulta agronomica</Text>
    <TextInput
      multiline
      placeholder="Escribi la consulta del productor..."
      placeholderTextColor={colors.text.secondary}
      style={styles.input}
    />
  </View>
);
}

const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flex: 1,
    padding: 20
  },
  title: {
    color: colors.text.primary,
    fontSize: 28,
    fontWeight: '700'
  },
  subtitle: {
    color: colors.text.secondary,
    fontSize: 16,
    marginBottom: 18
  },
  input: {
    backgroundColor: colors.surface.card,
    borderColor: colors.surface.border,
    borderRadius: 8,
    borderWidth: 1,
    color: colors.text.primary,
    minHeight: 140,
    padding: 14,
    textAlignVertical: 'top'
  }
});
```

AgroAidApp/app/(tabs)/mapa.tsx - CREAR placeholder

```
import { StyleSheet, Text, View } from 'react-native';

import { colors } from '@/constants/colors';

export default function MapaTab() {
  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Mapa colaborativo</Text>
      <Text style={styles.text}>
```

AgroAidApp - Dias 3 a 6

```
        Aca se integrara react-native-maps con eventos reportados por zona.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flex: 1,
    padding: 20
  },
  title: {
    color: colors.text.primary,
    fontSize: 24,
    fontWeight: '700',
    marginBottom: 8
  },
  text: {
    color: colors.text.secondary,
    fontSize: 16,
    lineHeight: 22
  }
});
```

AgroAidApp/app/(tabs)/historial.tsx - CREAR placeholder

```
import { StyleSheet, Text, View } from 'react-native';

import { colors } from '@/constants/colors';

export default function HistorialTab() {
  return (
    <View style={styles.screen}>
      <Text style={styles.title}>Historial</Text>
      <Text style={styles.text}>
        Las consultas guardadas se listaran aca en el Dia 6.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flex: 1,
    padding: 20
  },
  title: {
    color: colors.text.primary,
    fontSize: 24,
    fontWeight: '700',
    marginBottom: 8
  },
  text: {
```

AgroAidApp - Dias 3 a 6

```
        color: colors.text.secondary,  
        fontSize: 16,  
        lineHeight: 22  
      }  
    });
```

AgroAidApp/app/(tabs)/recetas.tsx - CREAM placeholder

```
import { StyleSheet, Text, View } from 'react-native';  
  
import { colors } from '@constants/colors';  
  
export default function RecetasTab() {  
  return (  
    <View style={styles.screen}>  
      <Text style={styles.title}>Recetas agronomicas</Text>  
      <Text style={styles.text}>  
        Aca se mostraran recomendaciones y descargas PDF cuando exista el servicio.  
      </Text>  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  screen: {  
    backgroundColor: colors.surface.background,  
    flex: 1,  
    padding: 20  
  },  
  title: {  
    color: colors.text.primary,  
    fontSize: 24,  
    fontWeight: '700',  
    marginBottom: 8  
  },  
  text: {  
    color: colors.text.secondary,  
    fontSize: 16,  
    lineHeight: 22  
  }  
});
```

AgroAidApp/app/(tabs)/perfil.tsx - CREAM

```
import { router } from 'expo-router';  
import { Pressable, StyleSheet, Text, View } from 'react-native';  
  
import { colors } from '@constants/colors';  
import { useAuth } from '@hooks/useAuth';  
import { useBranding } from '@hooks/useBranding';  
  
export default function PerfilTab() {  
  const { logout } = useAuth();
```

```
const { branding } = useBranding();

async function handleLogout() {
  await logout();
  router.replace('/login');
}

return (
  <View style={styles.screen}>
    <Text style={styles.title}>Perfil</Text>
    <Text style={styles.text}>{branding.footer_text}</Text>
    <Pressable onPress={handleLogout} style={styles.button}>
      <Text style={styles.buttonText}>Cerrar sesion</Text>
    </Pressable>
  </View>
);
}

const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flex: 1,
    padding: 20
  },
  title: {
    color: colors.text.primary,
    fontSize: 24,
    fontWeight: '700',
    marginBottom: 8
  },
  text: {
    color: colors.text.secondary,
    fontSize: 16,
    lineHeight: 22,
    marginBottom: 20
  },
  button: {
    alignItems: 'center',
    backgroundColor: colors.risk.critical,
    borderRadius: 8,
    minHeight: 48,
    justifyContent: 'center'
  },
  buttonText: {
    color: colors.text.inverse,
    fontSize: 16,
    fontWeight: '700'
  }
});
```

DIA 4

Este día conecta el tab de consulta con el backend. El endpoint exige token: debe funcionar después del login.

AgroAidApp/services/consultas.ts - CREAR

```
import { apiFetch } from '@services/api';

export type NivelRiesgo = 'BAJO' | 'MEDIO' | 'ALTO' | 'CRITICO' | string;

export type ConsultaResponse = {
  consulta_id: number;
  iteracion: number;
  max_iteraciones: number;
  completado: boolean;
  confianza: number;
  nivel_riesgo: NivelRiesgo;
  evidencia_suficiente: boolean;
  dominios_detectados: string[];
  justificacion: string;
  riesgos_detectados: string[];
  informacion_faltante: string[];
  preguntas_seguimiento: string[];
  debe_abstenerse: boolean;
  marco_regulatorio_aplicable: string[];
  requiere_profesional: boolean;
  evaluacion_final: string | null;
  verificacion_evidencia: Record<string, unknown> | null;
  se_abstuvo_final: boolean;
};

export type IniciarConsultaPayload = {
  consulta_inicial: string;
  tenant_slug?: string;
  lat?: number;
  lon?: number;
  umbral_confianza?: number;
  max_iteraciones?: number;
};

export function iniciarConsulta(payload: IniciarConsultaPayload) {
  return apiFetch<ConsultaResponse>('/api/consulta', {
    method: 'POST',
    body: JSON.stringify({
      tenant_slug: 'default',
      umbral_confianza: 80,
      max_iteraciones: 5,
      ...payload
    })
  });
}

export function responderConsulta(consultaId: number, respuesta: string) {
```

```
return apiFetch<ConsultaResponse>(`/api/consulta/${consultaId}/responder`, {
  method: 'POST',
  body: JSON.stringify({ respuesta })
});
}

export function getConsulta(consultaId: number) {
  return apiFetch<ConsultaResponse>(`/api/consulta/${consultaId}`);
}

export function descartarConsulta(consultaId: number) {
  return apiFetch<{ ok: boolean; consulta_id: number }>(
    `/api/consulta/${consultaId}`,
    { method: 'DELETE' }
  );
}
```

AgroAidApp/hooks/useConsulta.ts - CREAR

```
import { useCallback, useState } from 'react';

import {
  descartarConsulta,
  getConsulta,
  iniciarConsulta,
  responderConsulta,
  type ConsultaResponse,
  type IniciarConsultaPayload
} from '@services/consultas';

export function useConsulta() {
  const [consulta, setConsulta] = useState<ConsultaResponse | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const iniciar = useCallback(async (payload: IniciarConsultaPayload) => {
    setIsLoading(true);
    setError(null);
    try {
      const data = await iniciarConsulta(payload);
      setConsulta(data);
      return data;
    } catch (err) {
      const message =
        err instanceof Error ? err.message : 'No se pudo iniciar la consulta.';
      setError(message);
      throw err;
    } finally {
      setIsLoading(false);
    }
  }, []);

  const responder = useCallback(
    async (respuesta: string) => {
      if (!consulta) {
```

```

        return null;
      }

      setIsLoading(true);
      setError(null);
      try {
        const data = await responderConsulta(consulta.consulta_id, respuesta);
        setConsulta(data);
        return data;
      } catch (err) {
        const message =
          err instanceof Error ? err.message : 'No se pudo enviar la respuesta.';
        setError(message);
        throw err;
      } finally {
        setIsLoading(false);
      }
    },
    [consulta]
  );

  const cargar = useCallback(async (consultaId: number) => {
    setIsLoading(true);
    setError(null);
    try {
      const data = await getConsulta(consultaId);
      setConsulta(data);
      return data;
    } finally {
      setIsLoading(false);
    }
  }, []);

  const reset = useCallback(async () => {
    if (consulta) {
      await descartarConsulta(consulta.consulta_id).catch(() => undefined);
    }

    setConsulta(null);
    setError(null);
  }, [consulta]);

  return {
    consulta,
    error,
    isLoading,
    iniciar,
    responder,
    cargar,
    reset
  };
}

```

AgroAidApp/app/(tabs)/index.tsx - MODIFICAR - reemplazar contenido completo

```
import { useState } from 'react';
import {
  ActivityIndicator,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  View
} from 'react-native';

import { colors } from '@constants/colors';
import { useConsulta } from '@hooks/useConsulta';

export default function ConsultaTab() {
  const [texto, setTexto] = useState('');
  const [respuesta, setRespuesta] = useState('');
  const { consulta, error, isLoading, iniciar, responder, reset } = useConsulta();

  async function handleIniciar() {
    if (!texto.trim()) {
      return;
    }

    await iniciar({ consulta_inicial: texto.trim() });
  }

  async function handleResponder() {
    if (!respuesta.trim()) {
      return;
    }

    await responder(respuesta.trim());
    setRespuesta('');
  }

  return (
    <ScrollView contentContainerStyle={styles.screen}>
      <Text style={styles.title}>Consulta agronomica</Text>

      {!consulta ? (
        <>
          <TextInput
            multiline
            onChangeText={setTexto}
            placeholder="Describe el problema, cultivo, producto o riesgo..."
            placeholderTextColor={colors.text.secondary}
            style={styles.input}
            value={texto}
          />
          <Pressable
            disabled={isLoading || !texto.trim()}
            onPress={handleIniciar}
            style={[styles.button, (isLoading || !texto.trim()) && styles.disabled]}
          >
            {isLoading ? (
```

```

        <ActivityIndicator color={colors.text.inverse} />
      ) : (
        <Text style={styles.buttonText}>Iniciar consulta</Text>
      )}
    </Pressable>
  </>
) : (
  <View style={styles.panel}>
    <Text style={styles.label}>Riesgo: {consulta.nivel_riesgo}</Text>
    <Text style={styles.text}>{consulta.justificacion}</Text>

    {consulta.preguntas_seguimiento.map((pregunta) => (
      <Text key={pregunta} style={styles.question}>- {pregunta}</Text>
    ))}

    {!consulta.completado ? (
      <>
        <TextInput
          multiline
          onChangeText={setRespuesta}
          placeholder="Responde las preguntas de seguimiento..."
          placeholderTextColor={colors.text.secondary}
          style={styles.input}
          value={respuesta}
        />
        <Pressable
          disabled={isLoading || !respuesta.trim()}
          onPress={handleResponder}
          style={[styles.button, (isLoading || !respuesta.trim()) && styles.disabled]}
        >
          <Text style={styles.buttonText}>Enviar respuesta</Text>
        </Pressable>
      </>
    ) : (
      <Text style={styles.text}>{consulta.evaluacion_final}</Text>
    )}

    <Pressable onPress={reset} style={styles.secondaryButton}>
      <Text style={styles.secondaryButtonText}>Nueva consulta</Text>
    </Pressable>
  </View>
)}

  {error ? <Text style={styles.error}>{error}</Text> : null}
</ScrollView>
);
}

const styles = StyleSheet.create({
  screen: {
    backgroundColor: colors.surface.background,
    flexGrow: 1,
    gap: 14,
    padding: 20
  },
  title: {

```

```

color: colors.text.primary,
fontSize: 26,
fontWeight: '700'
},
input: {
  backgroundColor: colors.surface.card,
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  color: colors.text.primary,
  minHeight: 120,
  padding: 14,
  textAlignVertical: 'top'
},
button: {
  alignItems: 'center',
  backgroundColor: colors.brand.primary,
  borderRadius: 8,
  minHeight: 48,
  justifyContent: 'center'
},
disabled: {
  opacity: 0.55
},
buttonText: {
  color: colors.text.inverse,
  fontWeight: '700'
},
panel: {
  backgroundColor: colors.surface.card,
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  gap: 10,
  padding: 14
},
label: {
  color: colors.text.primary,
  fontSize: 16,
  fontWeight: '700'
},
text: {
  color: colors.text.secondary,
  lineHeight: 20
},
question: {
  color: colors.text.primary,
  lineHeight: 20
},
secondaryButton: {
  alignItems: 'center',
  backgroundColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  minHeight: 44,
  justifyContent: 'center'
}

```

AgroAidApp - Dias 3 a 6

```
    },  
    secondaryButtonText: {  
      color: colors.brand.primary,  
      fontWeight: '700'  
    },  
    error: {  
      color: colors.risk.critical  
    }  
  }  
});
```

DIA 5

Este día separa la UI del flujo de consulta en componentes reutilizables.

AgroAidApp/components/RiesgoChip.tsx - CREAR

```
import { StyleSheet, Text, View } from 'react-native';

import { colors } from '@constants/colors';

const riskColors: Record<string, string> = {
  BAJO: colors.risk.low,
  LOW: colors.risk.low,
  MEDIO: colors.risk.medium,
  MEDIUM: colors.risk.medium,
  ALTO: colors.risk.high,
  HIGH: colors.risk.high,
  CRITICO: colors.risk.critical,
  CRITICAL: colors.risk.critical
};

export function RiesgoChip({ nivel }: { nivel: string }) {
  const normalized = nivel.toUpperCase();
  const backgroundColor = riskColors[normalized] ?? colors.text.secondary;

  return (
    <View style={[styles.chip, { backgroundColor }]}>
      <Text style={styles.text}>{normalized}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  chip: {
    alignSelf: 'flex-start',
    borderRadius: 8,
    paddingHorizontal: 10,
    paddingVertical: 5
  },
  text: {
    color: colors.text.inverse,
    fontSize: 12,
    fontWeight: '700'
  }
});
```

AgroAidApp/components/ConsultaCard.tsx - CREAR

```
import { StyleSheet, Text, View } from 'react-native';

import { colors } from '@constants/colors';
```

```
import type { ConsultaResponse } from '@services/consultas';
import { RiesgoChip } from '@components/RiesgoChip';

type Props = {
  consulta: ConsultaResponse;
};

export function ConsultaCard({ consulta }: Props) {
  return (
    <View style={styles.card}>
      <View style={styles.header}>
        <Text style={styles.title}>Consulta #{consulta.consulta_id}</Text>
        <RiesgoChip nivel={consulta.nivel_riesgo} />
      </View>

      <Text style={styles.meta}>
        Iteracion {consulta.iteracion} de {consulta.max_iteraciones} - Confianza{' '}
        {consulta.confianza}%
      </Text>

      {consulta.justificacion ? (
        <Text style={styles.body}>{consulta.justificacion}</Text>
      ) : null}

      {consulta.riesgos_detectados.length > 0 ? (
        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Riesgos detectados</Text>
          {consulta.riesgos_detectados.map((riesgo) => (
            <Text key={riesgo} style={styles.item}>- {riesgo}</Text>
          ))}
        </View>
      ) : null}

      {consulta.preguntas_seguimiento.length > 0 ? (
        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Preguntas de seguimiento</Text>
          {consulta.preguntas_seguimiento.map((pregunta) => (
            <Text key={pregunta} style={styles.item}>- {pregunta}</Text>
          ))}
        </View>
      ) : null}

      {consulta.completado && consulta.evaluacion_final ? (
        <View style={styles.section}>
          <Text style={styles.sectionTitle}>Evaluacion final</Text>
          <Text style={styles.body}>{consulta.evaluacion_final}</Text>
        </View>
      ) : null}
    </View>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: colors.surface.card,
    borderColor: colors.surface.border,
```

AgroAidApp - Dias 3 a 6

```
        borderRadius: 8,
        borderWidth: 1,
        gap: 10,
        padding: 14
      },
      header: {
        alignItems: 'center',
        flexDirection: 'row',
        justifyContent: 'space-between',
        gap: 12
      },
      title: {
        color: colors.text.primary,
        flex: 1,
        fontSize: 18,
        fontWeight: '700'
      },
      meta: {
        color: colors.text.secondary,
        fontSize: 13
      },
      body: {
        color: colors.text.primary,
        fontSize: 15,
        lineHeight: 21
      },
      section: {
        gap: 6
      },
      sectionTitle: {
        color: colors.text.primary,
        fontWeight: '700'
      },
      item: {
        color: colors.text.secondary,
        lineHeight: 20
      }
    }
  });
```

AgroAidApp/components/RespuestaInput.tsx - CREAR

```
import { Pressable, StyleSheet, Text, TextInput, View } from 'react-native';

import { colors } from '@constants/colors';

type Props = {
  value: string;
  onChangeText: (value: string) => void;
  onSubmit: () => void;
  disabled?: boolean;
};

export function RespuestaInput({
  value,
  onChangeText,
```

```

    onSubmit,
    disabled
  }: Props) {
    const isDisabled = disabled || !value.trim();

    return (
      <View style={styles.wrapper}>
        <TextInput
          multiline
          onChangeText={onChangeText}
          placeholder="Responde con datos concretos: producto, dosis, cultivo, clima, zona..."
          placeholderTextColor={colors.text.secondary}
          style={styles.input}
          value={value}
        />
        <Pressable
          disabled={isDisabled}
          onPress={onSubmit}
          style={[styles.button, isDisabled && styles.buttonDisabled]}
        >
          <Text style={styles.buttonText}>Enviar respuesta</Text>
        </Pressable>
      </View>
    );
  }

  const styles = StyleSheet.create({
    wrapper: {
      gap: 10
    },
    input: {
      backgroundColor: colors.surface.card,
      borderColor: colors.surface.border,
      borderRadius: 8,
      borderWidth: 1,
      color: colors.text.primary,
      minHeight: 120,
      padding: 14,
      textAlignVertical: 'top'
    },
    button: {
      alignItems: 'center',
      backgroundColor: colors.brand.primary,
      borderRadius: 8,
      minHeight: 48,
      justifyContent: 'center'
    },
    buttonDisabled: {
      opacity: 0.55
    },
    buttonText: {
      color: colors.text.inverse,
      fontWeight: '700'
    }
  });

```

AgroAidApp/components/LoadingOverlay.tsx - CREAR

```
import { ActivityIndicator, StyleSheet, View } from 'react-native';

import { colors } from '@/constants/colors';

export function LoadingOverlay({ visible }: { visible: boolean }) {
  if (!visible) {
    return null;
  }

  return (
    <View style={styles.overlay}>
      <ActivityIndicator color={colors.text.inverse} size="large" />
    </View>
  );
}

const styles = StyleSheet.create({
  overlay: {
    ...StyleSheet.absoluteFillObject,
    alignItems: 'center',
    backgroundColor: 'rgba(22, 35, 22, 0.42)',
    justifyContent: 'center',
    zIndex: 10
  }
});
```

AgroAidApp/app/(tabs)/index.tsx - MODIFICAR - reemplazar contenido completo

```
import { useState } from 'react';
import { Pressable, ScrollView, StyleSheet, Text, TextInput, View } from 'react-native';

import { ConsultaCard } from '@/components/ConsultaCard';
import { LoadingOverlay } from '@/components/LoadingOverlay';
import { RespuestaInput } from '@/components/RespuestaInput';
import { colors } from '@/constants/colors';
import { useConsulta } from '@/hooks/useConsulta';

export default function ConsultaTab() {
  const [texto, setTexto] = useState('');
  const [respuesta, setRespuesta] = useState('');
  const { consulta, error, isLoading, iniciar, responder, reset } = useConsulta();

  async function handleIniciar() {
    if (!texto.trim()) {
      return;
    }

    await iniciar({ consulta_inicial: texto.trim() });
  }

  async function handleResponder() {
    if (!respuesta.trim()) {

```

```

        return;
    }

    await responder(respuesta.trim());
    setRespuesta('');
}

return (
    <View style={styles.container}>
        <LoadingOverlay visible={isLoading} />
        <ScrollView contentContainerStyle={styles.screen}>
            <Text style={styles.title}>Consulta agronomica</Text>

            {!consulta ? (
                <View style={styles.startPanel}>
                    <TextInput
                        multiline
                        onChangeText={setTexto}
                        placeholder="Describe el problema, cultivo, producto o riesgo..."
                        placeholderTextColor={colors.text.secondary}
                        style={styles.input}
                        value={texto}
                    />
                    <Pressable
                        disabled={isLoading || !texto.trim()}
                        onPress={handleIniciar}
                        style={[styles.button, (isLoading || !texto.trim()) && styles.disabled]}
                    >
                        <Text style={styles.buttonText}>Iniciar consulta</Text>
                    </Pressable>
                </View>
            ) : (
                <View style={styles.flow}>
                    <ConsultaCard consulta={consulta} />

                    {!consulta.completado ? (
                        <RespuestaInput
                            disabled={isLoading}
                            onChangeText={setRespuesta}
                            onSubmit={handleResponder}
                            value={respuesta}
                        />
                    ) : null}

                    <Pressable onPress={reset} style={styles.secondaryButton}>
                        <Text style={styles.secondaryButtonText}>Nueva consulta</Text>
                    </Pressable>
                </View>
            )}

            {error ? <Text style={styles.error}>{error}</Text> : null}
        </ScrollView>
    </View>
);
}
const styles = StyleSheet.create({

```

```
container: {
  flex: 1
},
screen: {
  backgroundColor: colors.surface.background,
  flexGrow: 1,
  gap: 14,
  padding: 20
},
title: {
  color: colors.text.primary,
  fontSize: 26,
  fontWeight: '700'
},
startPanel: {
  gap: 12
},
flow: {
  gap: 14
},
input: {
  backgroundColor: colors.surface.card,
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  color: colors.text.primary,
  minHeight: 150,
  padding: 14,
  textAlignVertical: 'top'
},
button: {
  alignItems: 'center',
  backgroundColor: colors.brand.primary,
  borderRadius: 8,
  minHeight: 48,
  justifyContent: 'center'
},
disabled: {
  opacity: 0.55
},
buttonText: {
  color: colors.text.inverse,
  fontWeight: '700'
},
secondaryButton: {
  alignItems: 'center',
  borderColor: colors.surface.border,
  borderRadius: 8,
  borderWidth: 1,
  minHeight: 44,
  justifyContent: 'center'
},
secondaryButtonText: {
  color: colors.brand.primary,
  fontWeight: '700'
},
```

AgroAidApp - Dias 3 a 6

```
    error: {  
      color: colors.risk.critical  
    }  
  });
```

DIA 6

El backend actual protege /api/historial con require_admin. Si entras con usuario comun puede devolver 401/403; para probar, usa una cuenta admin o ajusta el backend despues.

AgroAidApp/services/historial.ts - CREAR

```
import { apiFetch } from '@services/api';

export type HistorialItem = {
  consulta_id: number;
  consulta_inicial: string;
  fecha_consulta: string;
  se_abstuvo: boolean;
  nivel_riesgo: string;
  confianza_final: number;
  evidencia_suficiente: boolean;
  iteraciones: number;
  duracion_seg: number;
};

export type HistorialResponse = {
  page: number;
  per_page: number;
  total: number;
  total_pages: number;
  items: HistorialItem[];
};

export type HistorialFilters = {
  page?: number;
  per_page?: number;
  fecha_desde?: string;
  fecha_hasta?: string;
  nivel_riesgo?: string;
};

function buildQuery(filters: HistorialFilters = {}) {
  const params = new URLSearchParams();

  Object.entries(filters).forEach(([key, value]) => {
    if (value !== undefined && value !== null && String(value).trim() !== '') {
      params.set(key, String(value));
    }
  });

  const query = params.toString();
  return query ? `?${query}` : '';
}

export function getHistorial(filters: HistorialFilters = {}) {
  return apiFetch<HistorialResponse>(`/api/historial${buildQuery(filters)}`);
}

export function exportarHistorialCSV(filters: Omit<HistorialFilters, 'page' | 'per_page'> = {}) {
```

```
return apiFetch<string>(`
  /api/historial/export${buildQuery({ ...filters, format: 'csv' } as HistorialFilters & { format: string })}`
);
}
```

AgroAidApp/app/(tabs)/historial.tsx - MODIFICAR - reemplazar contenido completo

```
import { useEffect, useState } from 'react';
import {
  FlatList,
  Pressable,
  RefreshControl,
  StyleSheet,
  Text,
  View
} from 'react-native';

import { RiesgoChip } from '@components/RiesgoChip';
import { colors } from '@constants/colors';
import {
  getHistorial,
  type HistorialItem,
  type HistorialResponse
} from '@services/historial';

export default function HistorialTab() {
  const [data, setData] = useState<HistorialResponse | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function load(page = 1) {
    setIsLoading(true);
    setError(null);
    try {
      const response = await getHistorial({ page, per_page: 20 });
      setData(response);
    } catch (err) {
      setError(
        err instanceof Error
          ? err.message
          : 'No se pudo cargar el historial.'
      );
    } finally {
      setIsLoading(false);
    }
  }

  useEffect(() => {
    load();
  }, []);

  function renderItem({ item }: { item: HistorialItem }) {
    return (
      <View style={styles.card}>
        <View style={styles.cardHeader}>

```

```

        <Text style={styles.cardTitle}>Consulta #{item.consulta_id}</Text>
        <RiesgoChip nivel={item.nivel_riesgo} />
    </View>
    <Text style={styles.date}>{new Date(item.fecha_consulta).toLocaleString()}</Text>
    <Text style={styles.query}>{item.consulta_inicial}</Text>
    <Text style={styles.meta}>
        Confianza {item.confianza_final}% - Iteraciones {item.iteraciones}
    </Text>
</View>
);
}

return (
    <View style={styles.screen}>
        <View style={styles.header}>
            <Text style={styles.title}>Historial</Text>
            <Pressable onPress={() => load()} style={styles.reloadButton}>
                <Text style={styles.reloadText}>Actualizar</Text>
            </Pressable>
        </View>

        {error ? <Text style={styles.error}>{error}</Text> : null}

        <FlatList
            contentContainerStyle={styles.list}
            data={data?.items ?? []}
            keyExtractor={(item) => String(item.consulta_id)}
            refreshControl={
                <RefreshControl refreshing={isLoading} onRefresh={() => load()} />
            }
            renderItem={renderItem}
            ListEmptyComponent={
                !isLoading ? (
                    <Text style={styles.empty}>Todavía no hay consultas para mostrar.</Text>
                ) : null
            }
        />
    </View>
);
}

const styles = StyleSheet.create({
    screen: {
        backgroundColor: colors.surface.background,
        flex: 1,
        padding: 20
    },
    header: {
        alignItems: 'center',
        flexDirection: 'row',
        justifyContent: 'space-between',
        marginBottom: 14
    },
    title: {
        color: colors.text.primary,
        fontSize: 24,

```

```

        fontWeight: '700'
    },
    reloadButton: {
        borderColor: colors.surface.border,
        borderRadius: 8,
        borderWidth: 1,
        paddingHorizontal: 12,
        paddingVertical: 8
    },
    reloadText: {
        color: colors.brand.primary,
        fontWeight: '700'
    },
    list: {
        gap: 12,
        paddingBottom: 24
    },
    card: {
        backgroundColor: colors.surface.card,
        borderColor: colors.surface.border,
        borderRadius: 8,
        borderWidth: 1,
        gap: 8,
        padding: 14
    },
    cardHeader: {
        alignItems: 'center',
        flexDirection: 'row',
        justifyContent: 'space-between',
        gap: 10
    },
    cardTitle: {
        color: colors.text.primary,
        flex: 1,
        fontSize: 17,
        fontWeight: '700'
    },
    date: {
        color: colors.text.secondary,
        fontSize: 13
    },
    query: {
        color: colors.text.primary,
        lineHeight: 20
    },
    meta: {
        color: colors.text.secondary,
        fontSize: 13
    },
    error: {
        color: colors.risk.critical,
        marginBottom: 10
    },
    empty: {
        color: colors.text.secondary,
        marginTop: 24,

```

AgroAidApp - Dias 3 a 6

```
        textAlign: 'center'  
    }  
});
```

Checklist final

- Crear carpetas si no existen: app/(tabs), hooks, services, components.
- Si el redirect actual apunta a '/tabs', cambiarlo luego a '/(tabs)' para Expo Router.
- Instalar dependencias si faltan: npm install expo-secure-store @expo/vector-icons.
- Levantar backend en <http://localhost:8000> o ajustar EXPO_PUBLIC_API_BASE_URL.
- Probar login, consulta, respuesta de seguimiento y carga de historial con usuario admin.